

Øvinger 1+2+3 IDATT2104

Gruppe 14

Sander Rusten Berge, Usman Ghafoorzai, Jonas Grønskag Johansen, Sander Sandvik Nessa, Garv Sood

Øving til Lagmodell og Wireshark

Hensikten med øvingen er å bli kjent med Lagmodellen (forenklet 5-lag OSI) ved å analysere pakkefangst med Wireshark (WS).

0. Få en hensiktsmessig pakkefangst

Pakkefangst med Wireshark viser at det går en overveldende mengde pakker med mange ulike protokoller over trådløsnettet på campus, og sikkert også på hjemmenettet. Vi skal avgrense denne første undersøkelsen til en usikret HTTP-webseite.

- Start fangst i Wireshark: Når du starter Wireshark vises mange nettverkskort på din pc. Dobbeltklikk på nettverkskortet hvor det er aktivitet, mest sannsynlig Wi-Fi kortet.
- Gjør oppslag på *datakom.no* eller annen webside som bruker usikret HTTP (ikke kryptert overføring HTTPS)
- Sett display-filter i Wireshark ved å skrive HTTP. Feltet skifter farge til grønt når det er skrevet et lovlig filter. Nå skal bare et fåtall pakker vises, og hvis det er slik kan du stoppe videre pakkefangst.

1. Undersøk protokollene i pakkene

Wireshark viser ett pakkehode per linje for den pakken man studerer. Detaljene i pakkehodet vises ved å åpne denne linjen (>).

Når man markerer en protokoll-linje (pakkehode) viser Wireshark hvor stort pakkehodet er. Verdien kan leses nederst til venstre «på bunnlinjen».

Fyll inn tabell

- Hvilke protokoller brukes for HTTP-pakkene?
- Hvor hører disse protokollene til i Forenklet 5-lags OSI-modell?
- Hvor stort er pakkehodet for hver protokoll?

Lagets navn	Protokoll	Lag nummer i lagmodell	Størrelsen på pakkeheader (ant. byte)
Lenkelag	Ethernet 2	2	14
Nettverkslag	IPv4	3	20
Transportlag *)	TCP	4	20
Appl.lag **)	HTTP	5	7

*) TCP-SYN pakken har noen «options» som gjør dette pakkehodet noe større. SYN-pakkene inneholder ikke meldingsdata, men brukes bare for å etablere forbindelse

**) HTTP-meldingens størrelse avhenger av hvilken pakke man ser på, ulike parameterlengder

3. Hvordan vet mottaker hvilken protokoll (prosess) på laget over som er mottaker av nyttelasten i pakken?

Når en pakke kommer til mottaker skjer det motsatte av innpakking, pakkehodene blir skrelt av og innholdet (nyttelasten) sendt opp til laget over. Det kan være mange aktuelle

protokoller på laget over så derfor har avsender for enkelhets skyld lagt inn beskjed om hvilken protokoll som er mottaker. «Har lagt inn» betyr at det er et felt i pakkehodet hvor denne informasjonen ligger.

a) Hva heter dette protokollfeltet på de ulike lagene og hvilken verdi har det?

Lag	Gjeldende Protokoll	Datafelt navn	Datafelt Verdi	Mottaker protokoll
Lenkelag	Ethernet 2	Type	IPv4	IPv4
Nettverkslag	IPv4	Protocol	TCP	TCP
Transportlag	TCP	Destination port	443	HTTP

*) sender og mottaker bruker forskjellige Portnummer, webtjener lytter på port 80

4. Hvilken protokoll bruker navnetjenesten DNS på transportlaget?

DNS (applikasjonlaget) kan velge blant to typer tjenester på transportlaget, enten pålitelig overføring med TCP eller upålitelig (best effort) med UDP.

Start Wireshark og sett display-filter «DNS». Vent til det dukker opp noen DNS-pakker, f.eks etter bruk av Nettleseren på ny webside, eller ved å pinge en webside fra CMD.

a) Hvilken protokoll er det DNS her benytter på transportlaget?

TCP

5. Adresser

a) Hvilken konfigurering har egen PC?

Pakker går mellom likestilte lag som kan befinne seg på ulike maskiner. Man må kunne identifisere kilde/avsender og destinasjon/mottaker på de ulike lagene. Derfor inneholder pakkehodene slike adresser. Man finner setting for egen PC slik: *Windows: kjør Ledetekst (eller CMD)*

- `>ipconfig` som viser IP-info for nettverkslaget
- `>ipconfig /all` for mer detaljer, blant annet MAC-adresser

Fyll inn tabell med hvilke adresser er din egen PC konfigurert med

Adressefelt	Verdi
MAC-adresse	F4-D1-08-02-5A-8F
IPv4	10.22.23.94
Default gateway IPv4	fe80::5:73ff:fea0:6%14 10.22.20.1
Nettmaske	255.255.252.0

b) Hvilke mottakeradresser sender din PC til?

Wireshark viser mottakeradresser (dst) i pakkene som sendes. Bruk kommando *CMD>ping* på et par webadresse mens Wireshark fanger. Finn mottakeradresser (dst) i pakkene som sendes. Fyll inn tabell:

Ping på webadresse	Dst MAC (lenkelaget)	Dst IPv4 eller IPv6
Google.com	00:00:0c:9f:f0:04	142.250.74.174
Twitter.com	00:00:0c:9f:f0:04	104.244.42.65
Blackboard.com	00:00:0c:9f:f0:04	20.85.72.68
Datakom.no	00:00:0c:9f:f0:04	20.100.42.130
ChatGPT.com	00:00:0c:9f:f0:04	140.82.121.3

c) Hvilken enhet er det som har MAC-adressen som pakken sendes til?

For å finne/verifisere svaret kan man undersøke ARP-tabell på egen PC. Denne tabellen viser en oversikt som knytter MAC-adresser til aktuelle IP-adresser. Her vil det vises en «kjent» IP-adresse for denne MAC-adressen. Slik søker man egen ARP-tabell: *Cmd>arp -a*

Og enheten som har MAC-adressen hvor pakkene sendes til er:

```
Interface: 10.22.22.250 --- 0xe
  Internet Address      Physical Address      Type
  10.22.20.1            00-00-0c-9f-f0-04    dynamic
  224.0.0.22            01-00-5e-00-00-16    static
  224.0.0.251           01-00-5e-00-00-fb    static
  224.0.0.252           01-00-5e-00-00-fc    static
  255.255.255.255       ff-ff-ff-ff-ff-ff    static

Interface: 192.168.56.1 --- 0x14
  Internet Address      Physical Address      Type
  192.168.56.255        ff-ff-ff-ff-ff-ff    static
  224.0.0.22            01-00-5e-00-00-16    static
  224.0.0.251           01-00-5e-00-00-fb    static
  224.0.0.252           01-00-5e-00-00-fc    static
  255.255.255.255       ff-ff-ff-ff-ff-ff    static
```

Den øverste adressen, 10.22.20.1

Øving 2: DNS, Epost og Web

Del 1: DNS

Det er flere siktemål med denne DNS-øvingen.

- Det første målet er å bli kjent med konfigurering på din egen PC. Du skal finne adressen til din lokale navnetjener og hva den heter. Her brukes kommandovindu på PC (ledetekst eller CMD på Windows).
- Videre skal du gjøre spesifikke DNS-oppslag hvor ett domenenavn brukes for ulike tjenester, web og epost, dette for å undersøke typefeltet i Ressursrecord.
- Til slutt skal Wireshark brukes til å finne levetiden for svar på DNS-oppslag, dvs hvor lenge de kan mellomlagres i cache.

1. Hva heter din lokale navnetjener? Vi søker etter domenenavnet.

Steg 1: Finn hvilken/hvilke DNS-tjenere din PC er konfigurert med når du er *koplet på campusnettet*.

Start Ledetekst/CMD og gi kommando: `CMD>ipconfig /all`. Merk: Uten tilføyelsen `/all` vil bare de mest sentrale parameterne vises, og ikke DNS-adresser.

SVAR: (Bruk gjerne skjermsklipp)

```
DNS Servers . . . . . : 2001:700:300::200
                        2001:700:300::201
                        129.241.0.200
                        129.241.0.201
```

Steg 2: Når dette DNS-oppslaget gjøres med tilkopling til NTNU-nettet vil det vises at jeg kan velge blant *fire* ulike IP-adresser til lokale navnetjenere, to IPv4 og to IPv6. Bruk `CMD>nslookup <IP>` til å finne navnet til den første IPv4-adressen.

SVAR:

```
C:\Users\jojoh>nslookup 129.241.0.200
Server:  dnscache1.ntnu.no
Address:  2001:700:300::200

Name:     dnscache1.ntnu.no
Address:  129.241.0.200
```

Merk: Et tjenernavn kan være registrert både med IPv4- og IPv6 adresse men allikevel være på samme maskin.

2. Finn IP-adressen til eposttjener for NTNU

Domenenavnet til NTNU er `ntnu.no`. Dette navnet brukes jo blant annet for adressering i epost og web, og disse tjenestene er plassert på ulike maskiner av kapasitetsmessige årsaker. Dermed må et

domenenavn kunne ha flere IP-adresser, avhengig av hvilke tjenester man bruker. Hvert ressursrecord i DNS har derfor angitt hvilken tjenestetype dette gjelder for.

Et forbehold: NTNU kjører store tjenester fordelt på flere serverer for redundans og lastbalansering. Når vi i oppgaven her søker på adresser og navn på tjenere så viser ikke dette hele bildet av NTNUs serverpark og hvordan de er knyttet sammen. Svarene kan derfor variere fra en PC til en annen.

Steg 1: Søke med nslookup på bestemt type ressursrecord (RR)

Det er 15-20 talls tjenestetyper av RR, de vanligste er

- A (IPv4)
- AAAA (IPv6)
- MX (Mail Exchange)
- NS (Name Server)
- CNAME (Alias, et navn som peker til et annet navn)
- PTR (Pointer)

Sett MX (for «Mail eXchange») og søk på ntnu.no i ledeteksten:

```
C:\Users\olav>nslookup
Default Server:  dnscache1.ntnu.no
Address:  2001:700:300::200

> set type=mx
>
```

Når man søker på ntnu.no med type MX vil man få *navnet* på en ny tjener til svar. Hva er navnet til NTNUs eposttjener?

SVAR:

```
C:\Users\jojoh>nslookup
Default Server:  dnscache1.ntnu.no
Address:  2001:700:300::200

> set type=mx
> ntnu.no
Server:  dnscache1.ntnu.no
Address:  2001:700:300::200

Non-authoritative answer:
ntnu.no MX preference = 10, mail exchanger = mx.ntnu.no
```

Nå må man gjøre et nytt oppslag for å spørre etter IP-adresse for denne navngitte tjeneren. Endre til «set type= AAAA» for å finne IPv6 til eposttjener.

Hva IPv6 til NTNUs eposttjener?

SVAR:

```
> set type=AAAA
> mx.ntnu.no
Server:  dnscache1.ntnu.no
Address:  2001:700:300::200

Non-authoritative answer:
Name:     mx.ntnu.no
Address:  2001:700:300:3::67
```

3. Finn IP-adressen og det «egentlige» navnet til tjeneren for «ntnu.no»

Gjør et nslookup-søk med type AAAA på «ntnu.no». Kjør så et nytt søk på denne IPv6-adressen, da får man navnet på tjeneren som er knyttet til denne adressen. Merk: Her vil man finne at tjeneren har et annet navn enn «ntnu.no». Det betyr at *ntnu.no* er «merkenavnet» på Internett, men at de ulike tjenerne bak dette merkenavnet har sine egne navn.

Hva er IPv6 til ntnu.no, og hva er navnet til denne tjeneren?

SVAR:

```
> 2001:700:300::200
Server:  dnscache1.ntnu.no
Address:  2001:700:300::200

Name:     dnscache1.ntnu.no
Address:  2001:700:300::200
```

Del 2 - EPOST

Hensikten med denne øvingen er å vise hvor enkelt det er å kople seg på en epost-tjener for å sende en epost, og med det vise hvordan SMTP-dialogen mellom klient og tjener fungerer.

Det skal kjøres en manuell Telnet-sesjon mot eposttjeneren på NTNU som om man selv var en ordinær epostklient. Denne sesjonen er ikke ment å fullføres, altså sende en epost, men hvis man allikevel forsøker på det så skal det uansett IKKE benyttes annen avsenderadresse enn sin egen legitime epostadresse. Det vil i så fall kunne gi meget uønskede konsekvenser for avsender.

1. Åpne en telnet-sesjon: `CMD>telnet`
2. Opprett forbindelse mot NTNUs eposttjener: `o mx.ntnu.no 25`
(Betyr open TCP mot port 25 som SMTP lytter på)
3. Det opprettes en TCP-sesjon og Tjeneren presenterer seg med kode 220, tjenesten er klar
4. Neste steg i dialogen er at klienten skal presentere seg, men for denne testen hopper man over det og skriver bare `<mail from:xx>`
5. Tjeneren gir da en feilmelding og forteller deg hva du egentlig skulle ha sendt.

Oppgave: Hva er det SMTP svarer i linje 6?

SVAR:

```
mail from:12
503 5.5.1 Error: send HELO/EHLO first
```

Dette er en utmerket demonstrasjon av dialogen mellom klient og tjener, og hvor enkelt det er å kople seg på en tjener. Det kreves ingen pålogging. Tilkoplingen kan riktignok krypteres med TLS ved å kople seg på port 587, men det er bare for å ha konfidensiell forbindelse mellom klient og tjener.

I teknisk forstand er det strengt tatt ikke noe som heter «falske eposter», men vi har epost med uønsket innhold, falsk avsender osv. Derfor gjør epost-tjenere en stor jobb med å filtrere innholdet på svartelistet IP-adresser og nøkkeltekst i innholdet. For å gi en indikasjon på omfanget så får NTNU omtrent 1 million eposter daglig, og litt forsiktig anslått er ca 80% av dette uønsket epost.

Del 3 - Webtjenesten og HTTP

Hensikten med denne øvingen å forstå hvordan HTTP fungerer, nærmere bestemt hvordan headerlinjer styrer dialogen mellom klient og tjener. Dette skal illustreres med Wireshark pakkefangst av HTTP/ 1.1. Vi kan ikke gjøre en tilsvarende undersøkelse av **HTTPS** fordi den har innhold kryptert med TLS. TLS vil bli undersøkt nærmere i neste øving.

A. Innholdet av HTTP-meldinger

I første del av øvingen benyttes det en ferdig Wireshark-pakkefangst fra et oppslag på *datakom.no* (hentes på Blackboard). Studer innholdet i pakke nr 13, respons fra HTTP-tjener. Denne pakken er siste del i overføringen av hele index.html fila

Når ble denne pakkefangsten tatt?

- **Svar:** 12. januar 2021 09:31:28 UTC

Når var denne index-fila sist endret?

- **Svar:** `Last-Modified: Wed, 23 Sep 2020 13:02:56 GMT\r\n`

Hvor mange TCP-segementer ble benyttet for å laste ned index-fila, og hvor store var de?

- **Svar:**

Brukte 5 TCP-segementer for å laste ned index-fila

5	0.001212	192.168.0.119	129.241.162.40	HTTP	673 GET / HTTP/1.1
6	0.001246	129.241.162.40	192.168.0.119	TCP	66 80 → 55599 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1460 SACK_PERM WS=128
7	0.001296	192.168.0.119	129.241.162.40	TCP	54 55599 → 80 [ACK] Seq=1 Ack=1 Win=262656 Len=0
8	0.001983	129.241.162.40	192.168.0.119	TCP	60 80 → 55598 [ACK] Seq=1 Ack=620 Win=30464 Len=0
9	0.002206	129.241.162.40	192.168.0.119	TCP	1514 80 → 55598 [ACK] Seq=1 Ack=620 Win=30464 Len=1460 [TCP PDU reassembled in 13]
10	0.003125	129.241.162.40	192.168.0.119	TCP	1514 80 → 55598 [ACK] Seq=1461 Ack=620 Win=30464 Len=1460 [TCP PDU reassembled in 13]
11	0.003125	129.241.162.40	192.168.0.119	TCP	1514 80 → 55598 [ACK] Seq=2921 Ack=620 Win=30464 Len=1460 [TCP PDU reassembled in 13]
12	0.003125	129.241.162.40	192.168.0.119	TCP	1514 80 → 55598 [ACK] Seq=4381 Ack=620 Win=30464 Len=1460 [TCP PDU reassembled in 13]
13	0.003125	129.241.162.40	192.168.0.119	HTTP	1171 HTTP/1.1 200 OK (text/html)

Kolonnen etter http og TCP forteller oss om størrelsen på pakkene sendt. Her ser vi at TCP pakkene var på størrelsene 66, 54, 60, 1514, 1514, 1514 bytes.

I pakke nr 13 viser Wireshark svaret fra tjener på klientens tidligere request i linje 5 om å bruke *vedvarende forbindelse*. Formatet kalles «kolonseparert navn/verdi-par». Hvordan er denne utvekslingen?

- **Svar:**

Klient spurte (i linje 5): Connection: keep-alive

Tjener svarer: Keep-Alive: timeout=5, max=100

Og dette svaret betyr: «timeout» er heltallsverdien for hvor lenge en tjeneren tillater at noen er koblet på uten å sende noe data før den avslutter forbindelsen. I dette tilfellet må det ta 5 sekunder der ingen data er blitt sendt for at tjener skal lukke forbindelsen. «max» er et heltall som forteller oss om hvor mange forespørsler som kan bli sendt på denne forbindelsen før tjeneren avslutter forbindelsen, i dette tilfellet 100.

HTTP gjøres tilstandsfull ved bruk av cookies. Hva betyr det at en forbindelse er tilstandsfull? Hvordan brukes cookies for å oppnå dette i dialogen mellom klient og tjener?

- **Svar:** Tilstandsløs betyr at den ikke har noen hukommelse. Tilstandsfull derimot betyr bare at vi har en hukommelse. En måte å få til dette er ved cookies (informasjons-kapsler). Cookies er små pakker som inneholder informasjon om en eller flere tilstander. Disse blir opprettet av tjeneren og lagret på klienten sin maskin. Nå når klienten sender en forespørsel til tjeneren blir disse pakkene også sendt med.

Det er unødvendig å laste ned uendrede filer som allerede finnes i mellomlager. Hvordan oppnår HTTP dette i dialog mellom klient og tjener?

Merk1: Dersom man sletter mellomlagrede filer så kan man kunne bruke Wireshark for å studere TCP-segmenter for nedlasting av store filer.

Merk 2: PHP-genererte html-filer vil alltid lastes ned fordi de er dynamisk genererte, alltid nye versjoner.

- **Svar:** Objekter fra web blir lagret i mellomlageret (cache). Når vi besøker en nettside vi allerede har vært på sendes det et spørsmål fra klient til tjener som spør om nettsiden har blitt oppdatert etter sist besøk. Dersom den ikke har det bruker klienten den allerede eksisterende nettsiden, dersom den er blitt endret blir den nyeste versjonen av nettsiden sendt.

Finn en pakke med HTTP-GET. Høyreklikk på denne pakken og velg Follow/HTTP-stream. Hva skjer?

Svar:

Når vi ber om å se hele http-streamen til en pakke får vi mulighet til å se alle forespørsler som blir gjort fra vår maskin til tjeneren. Her ser vi at vi gjør 2 forespørsler. Den første er at vi ber om selve HTML-koden, og den andre er når vi ber om CSS filen.

Pakke 1 og 2 inneholder begge et TCP-SYN-flagg. Observer at ulike TCP-portnummer knyttes til disse. Hvilken effekt har dette for webtjenesten?

Svar:

Pakke 1 sender fra port 55598 til port 80 hos tjener, og pakke 2 sender fra port 55599. Ved å sette bestemt port har vi muligheten til å høre på samme port med to ulike klienter uten at informasjonen blir blandet, da tjener vet hvilken port hver informasjonskapsel tilhører.

Labøving til uke 4 - Bli kjent med TCP og TLS ved bruk av Wireshark

Del 1 undersøker hvordan TCP etablerer en forbindelse med Three-way handshake (3WHS), kjent som Syn, SYN-ACK og ACK. Deretter undersøkes hvordan data overføres pålitelig ved å bruke sekvensnummer og kvitteringsnummer.

Del 2 undersøker hvordan HTTPS etableres, dvs at TLS setter opp en kryptert forbindelse for HTTP-kommunikasjon.

Øvingen – pakkefangst med Wireshark

Start Wireshark og gjør et oppslag på Blackboard uten at man er pålogget fra før. Som regel dukker det opp så mange pakker i Wireshark at det er vanskelig å finne akkurat denne sesjonen vi ønsker å analysere. Vi er på jakt etter pakker som viser «ntnu-blackboard.com» i pakkelinjen

- a) For å se hvilke TLS-sesjoner som har vært i bruk kan man først sette display-filer **tls**

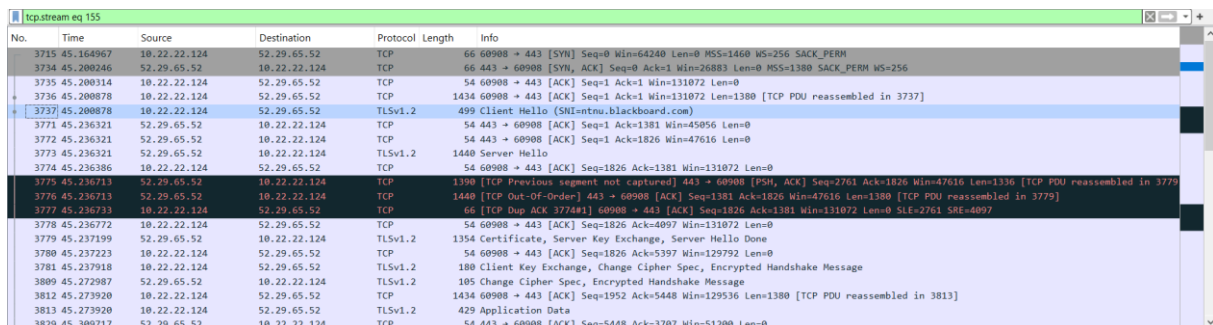
Det kan fortsatt være mange pakker, PC-en er aktiv med masse online-aktiviteter.

- b) Bruk et mer spesifikt display-filer:
tls.handshake.extensions_server_name=="ntnu.blackboard.com"

Nå skal det vises bare en eller et fåtall «Client Hello» fra vår sesjon.

En nettleser kan sette opp parallelle sesjoner for å bedre responstiden. Vi ønsker å undersøke en særskilt strøm. Høyreklikk på en av pakkene med første Client Hello og velg **Follow / TCP-stream**

Del 1: Undersøke TCP oppkopling og dataoverføring



No.	Time	Source	Destination	Protocol	Length	Info
3715	45.164967	10.22.22.124	52.29.65.52	TCP	66	60908 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
3734	45.200246	52.29.65.52	10.22.22.124	TCP	66	443 → 60908 [SYN, ACK] Seq=0 Ack=1 Win=26883 Len=0 MSS=1380 SACK_PERM WS=256
3735	45.200314	10.22.22.124	52.29.65.52	TCP	54	60908 → 443 [ACK] Seq=1 Ack=1 Win=131072 Len=0
3736	45.200878	10.22.22.124	52.29.65.52	TCP	1434	60908 → 443 [ACK] Seq=1 Ack=1 Win=131072 Len=1380 [TCP PDU reassembled in 3737]
3737	45.200878	10.22.22.124	52.29.65.52	TLSv1.2	499	Client Hello (SNI=ntnu.blackboard.com)
3771	45.236321	52.29.65.52	10.22.22.124	TCP	54	443 → 60908 [ACK] Seq=1 Ack=1381 Win=45056 Len=0
3772	45.236321	52.29.65.52	10.22.22.124	TCP	54	443 → 60908 [ACK] Seq=1 Ack=1826 Win=47616 Len=0
3773	45.236321	52.29.65.52	10.22.22.124	TLSv1.2	1440	Server Hello
3774	45.236386	10.22.22.124	52.29.65.52	TCP	54	60908 → 443 [ACK] Seq=1826 Ack=1381 Win=131072 Len=0
3775	45.236713	52.29.65.52	10.22.22.124	TCP	1390	[TCP Previous segment not captured] 443 → 60908 [PSH, ACK] Seq=2761 Ack=1826 Win=47616 Len=1336 [TCP PDU reassembled in 3779]
3776	45.236713	52.29.65.52	10.22.22.124	TCP	1440	[TCP Out-Of-Order] 443 → 60908 [ACK] Seq=1381 Ack=1826 Win=47616 Len=1380 [TCP PDU reassembled in 3779]
3777	45.236713	10.22.22.124	52.29.65.52	TCP	66	[TCP Dup ACK 3774#1] 60908 → 443 [ACK] Seq=1826 Ack=1381 Win=131072 Len=0 SLE=2761 SRE=4097
3778	45.236772	10.22.22.124	52.29.65.52	TCP	54	60908 → 443 [ACK] Seq=1826 Ack=4097 Win=131072 Len=0
3779	45.237199	52.29.65.52	10.22.22.124	TLSv1.2	1354	Certificate, Server Key Exchange, Server Hello Done
3780	45.237223	10.22.22.124	52.29.65.52	TCP	54	60908 → 443 [ACK] Seq=1826 Ack=5397 Win=129792 Len=0
3781	45.237918	10.22.22.124	52.29.65.52	TLSv1.2	180	Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
3809	45.272987	52.29.65.52	10.22.22.124	TLSv1.2	105	Change Cipher Spec, Encrypted Handshake Message
3812	45.273920	10.22.22.124	52.29.65.52	TCP	1434	60908 → 443 [ACK] Seq=1952 Ack=5448 Win=129536 Len=1380 [TCP PDU reassembled in 3813]
3813	45.273920	10.22.22.124	52.29.65.52	TLSv1.2	429	Application Data
3829	45.309717	52.29.65.52	10.22.22.124	TCP	54	443 → 60908 [ACK] Seq=5448 Ack=3707 Win=51200 Len=0

A. TCP oppkopling (3WHS)

Identifiser de tre oppstartspakkene fremst i listen som har flaggene SYN, SYN-ACK og ACK.

Fyll inn tabeller:

SYN-pakke 1 fra klient til tjener

Innhold	Verdi	Merknad
Flagg	0x002 (SYN)	Oppstartspakke
Destination Port adr	443	Velkjent portnummer HTTPS
Source Port adr	60908	Dynamisk/ kortlivde portnummer
Rel. seq. nr	0	ISN-nummer er 32 bit, her viser Wireshark et relativt 0

Rel. ack. nr	0	
Options	MSS=1460, WS=256 SACK_PERM	Eks. SACK=Selective Repeat v. retransmisjon
TCP-Pakkehode størrelse	20	Vanlige segmenter har 20 byte pakkehode

SYN-ACK pakke 2: fra Tjener til klient

Innhold	Verdi
Flagg	0x012 (SYN, ACK)
Destination Port adr	60908
Source Port adr	443
Rel. seq. nr	0
Rel. ack. nr	1
Options	MSS=1460, WS=256 SACK_PERM
TCP-pakkehode størrelse	20

ACK pakke 3: fra Klient til tjener

Innhold	Verdi
Flagg	0x010 (ACK)
Destination Port adr	443
Source Port adr	60908
Rel. seq. nr	1
Rel. ack. nr	1
Options	Ingen options
TCP-pakkehode størrelse	20

B. TCP dataoverføring

Du skal undersøke hvordan data kvitteres slik at overføringen blir pålitelig. Dette kan egentlig startes hvor som helst i en TCP-assatrøm.

TIPS 1: Du kan fortsatt bruke «Follow /TCP-stream» som filtrerer vekk alle pakker som ikke gjelder denne sesjonen.

Tips 2: Det vil gå en datastrøm i begge retninger og som kvitteres hver for seg.

Fyll inn tabeller. Det benyttes relativ nummerering på sekvens- og kvitteringsnummer (det nummeret som Wireshark viser).

1. Første client hello data-pakke fra klient

Innhold	Verdi
Sekv. nr	4186765137
Ack. nr	666864575
Nyttelast (byte)	445

2. Første svar fra tjener (dette kan være en pakke uten nyttelast, bare med ACK og kvitteringsnummer)

Innhold	verdi
Sekv. nr	666864575
Ack. nr	4186765137
Nyttelast (byte)	Ingen nyttelast

Undersøk videre pakkeflyt og dann deg et bilde av hvordan sekvens- og kvitteringsnummer endres.
Merk at kvitteringsnummer som returneres er lik mottatt <sekvensnummer + nyttelast>.

Del 2: TLS meldingsutveksling

Start med å lese den følgende teksten under. Deretter skal du bruke Wireshark til å se at TLS faktisk følger punktene 1-5 i praksis også.

Teksten er hentet fra [engelsk Wikipedia](#).

1. The handshake begins when a client connects to a TLS-enabled server requesting a secure connection and the client presents a list of supported cipher suites (ciphers and hash functions).
2. From this list, the server picks a cipher and hash function that it also supports and notifies the client of the decision.
3. The server usually then provides identification in the form of a digital certificate. The certificate contains the server name, the trusted certificate authority (CA) that vouches for the authenticity of the certificate, and the server's public encryption key.
4. The client confirms the validity of the certificate before proceeding.
5. To generate the session keys used for the secure connection, the client either:
 - a. encrypts a random number (PreMasterSecret) with the server's public key and sends the result to the server (which only the server should be able to decrypt with its private key); both parties then use the random number to generate a unique session key for subsequent encryption and decryption of data during the session
 - b. uses Diffie–Hellman key exchange (or its variant elliptic-curve DH) to securely generate a random and unique session key for encryption and decryption that has the additional property of forward secrecy: if the server's private key is disclosed in future, it cannot be used to decrypt the current session, even if the session is intercepted and recorded by a third party.
6. This concludes the handshake and begins the secured connection, which is encrypted and decrypted with the session key until the connection closes. If any one of the above steps fails, then the TLS handshake fails, and the connection is not created.

When secured by TLS, connections between a client (e.g., a web browser) and a server (e.g., wikipedia.org) should have one or more of the following properties:

- a) The connection is private (or has **confidentiality**) because a symmetric-key algorithm is used to encrypt the data transmitted.
- b) The identity of the communicating parties can be **authenticated** using public-key cryptography. This authentication is required for the server and optional for the client.
- c) The connection is reliable (or has **integrity**) because each message transmitted includes a message integrity check using a message authentication code to prevent undetected loss

Praktisk oppgave

Steg 1) Client Hello

Finn *cipher suite*-feltet og identifiser listen med suiter som støttes. Identifiser innholdet i Cipher Suite (støttede algoritmer) og verifiser at det inngår forkortelser som SHA, AES og RSA. Minn deg selv på hva disse forkortelsene betyr.

SHA (Secure Hashing Algorithm), AES (Advanced Encryption Standard), RSA (Rivest-Shamir-Adleman)

I «Client Hello» pakken finner vi i cypher suit-feltet at alle de overnevnte krypteringsalgoritmene finnes.

Steg 2) Server Hello

Finn cipher suite og noter hvilken suite som er valgt. Sjekk at denne suiten står i listen fra Client Hello

I «Server Hello» pakken finner vi at «TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 (0xc02f)» er valgt.

Steg 3) Certificate, server key exchange, server done

Observer at serveren her sender både sertifikater og hvilke mulighet som tilbys for oppretting av sesjonsnøkler. Dette (eller disse) sertifikatene er det klienten verifiserer. **OBS: Denne pakken sedens bare ved etablering av forbindelsen, ikke ved videre nedlastinger av objekter fra websiden.** Derfor valgte man det første client Hello i Wireshark

Steg 4 og 5) Client Key exchange/Change cipher Specs.

Etter dette benyttes kryptert overføring og videre HTTP-kommunikasjon er ikke lesbar for oss i pakkefangsten.